Lecture 8 · Theme 2: Learning representations

Backpropagation: notes

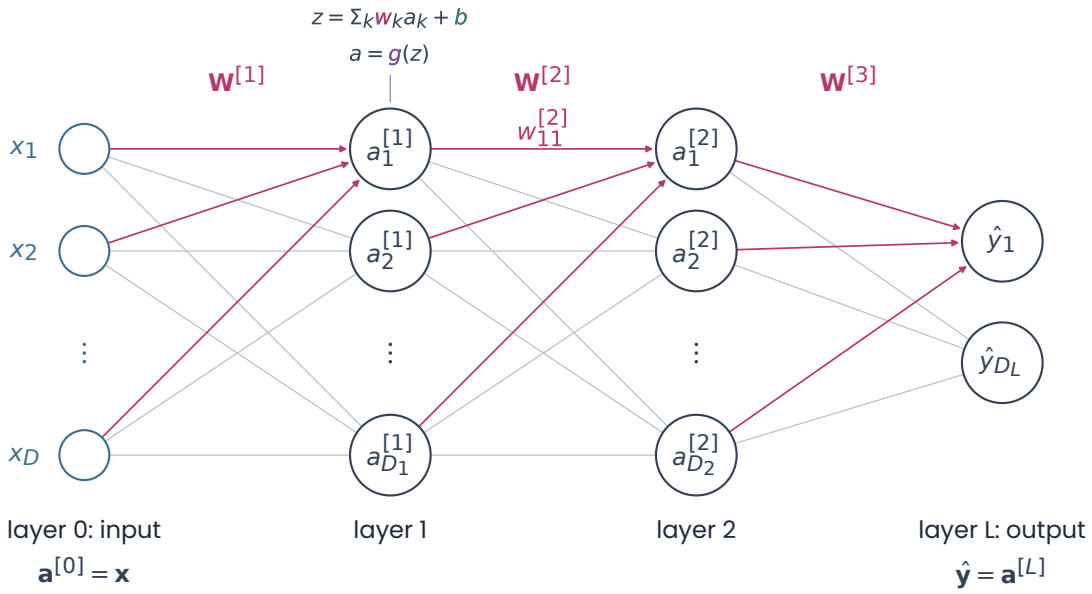
Thomas Serre · Professor of Cognitive & Psychological Sciences · Carney Institute for Brain Science, Brown University · Thursday, October 8 · Fall 2026

You will not be asked to derive backpropagation in this course, although students with a background in computer science should be able to follow the derivation. Everyone should be able to run it by hand on a small network and to read the equations below.

These notes collect the lecture's equations and its worked example in one place, with the same symbols as the slides.

A multilayer perceptron and its notation

A multilayer perceptron (MLP) stacks layers of model neurons. Each unit computes a weighted sum of the previous layer's outputs, adds a bias, and passes the result through a nonlinearity g. The outputs of one layer are the inputs of the next. The figure shows an MLP with two hidden layers. The magenta connections are the weights into the first unit of each layer, that is, one row of each weight matrix. The XOR network of the worked example below is the smallest case: two inputs, one hidden layer of two units, and one output, so $L = 2$.



A network has layers $l = 0, 1, \ldots, L$. Layer 0 is the input and layer L the output. Layer l has D_l units, its width, and the input has $D_0 = D$ entries. Vectors are bold column vectors. Each row of a layer's weight matrix holds the weights into one unit, so the forward pass needs no transpose.

Symbol	Name	Shape
$\boldsymbol{x} = \boldsymbol{a}^{[0]}$	input	D_0
$\boldsymbol{W}^{[l]}$	weight matrix of layer l (row j: the weights into unit j)	$D_l \times D_{l-1}$
$w_{jk}^{[l]}$	weight from unit k of layer $l-1$ to unit j of layer l	scalar
$\boldsymbol{b}^{[l]}$	bias of layer l, one per unit	D_l
$\boldsymbol{z}^{[l]}$	weighted sum (pre-activation) of layer l	D_l
g, g'	activation function and its slope, applied to each entry	–
$\boldsymbol{a}^{[l]}$	activation of layer l	D_l
$\hat{\boldsymbol{y}} = \boldsymbol{a}^{[L]}$	prediction (output of the last layer)	D_L
$\boldsymbol{y}$	target	D_L
ℓ	loss on one example	scalar
$\mathcal{L}$	loss averaged over the examples used for one step	scalar
$\boldsymbol{\delta}^{[l]}$	error of layer l, $\partial\mathcal{L}/\partial\boldsymbol{z}^{[l]}$	D_l
η	learning rate	scalar
$\odot$	entry-by-entry product of two vectors of the same length	–

A *gradient* $\partial\mathcal{L}/\partial\boldsymbol{W}^{[l]}$ has the shape of $\boldsymbol{W}^{[l]}$: one entry per weight, each saying how much the loss would change if that weight were nudged. For a single example, $\mathcal{L} = \ell$. With several examples, $\mathcal{L}$ is the average of their losses, and its gradient is the average of their gradients.

Forward pass

Set $\boldsymbol{a}^{[0]} = \boldsymbol{x}$. Then, for $l = 1, \ldots, L$,

$$\boldsymbol{z}^{[l]} = \boldsymbol{W}^{[l]}\boldsymbol{a}^{[l-1]} + \boldsymbol{b}^{[l]}, \qquad \boldsymbol{a}^{[l]} = g\big(\boldsymbol{z}^{[l]}\big),$$

and the prediction is $\hat{\boldsymbol{y}} = \boldsymbol{a}^{[L]}$. Each unit is a model neuron: unit j of layer l computes $z_j^{[l]} = \sum_k w_{jk}^{[l]} a_k^{[l-1]} + b_j^{[l]}$, and g acts on each entry of $\boldsymbol{z}^{[l]}$ separately. The hidden layers here use $\mathrm{ReLU}(z) = \max(0, z)$, whose slope $g'(z)$ is 1 when $z > 0$ and 0 otherwise. The output layer uses a sigmoid σ for one yes/no output, or a softmax for one output per class. The loss is the cross-entropy, which for one sigmoid output is

$$\ell = -\big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big].$$

When $y = 1$ it reduces to $-\log \hat{y}$. The forward pass keeps every $\boldsymbol{z}^{[l]}$ and $\boldsymbol{a}^{[l]}$, because the backward pass needs them.

Backward pass

Output error. With a sigmoid or softmax output and the cross-entropy loss, the error of the output layer is prediction minus target,

$$\boldsymbol{\delta}^{[L]} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{[L]}} = \hat{\boldsymbol{y}} - \boldsymbol{y}.$$

For one sigmoid output, $\partial \ell / \partial \hat{y} = (\hat{y} - y)/\big(\hat{y}(1-\hat{y})\big)$ and $\sigma'(z) = \hat{y}(1-\hat{y})$, and their product is $\hat{y} - y$. This is the same result as for logistic regression, earlier in the lecture.

Error sent backward. For $l = L-1, \ldots, 1$,

$$\boldsymbol{\delta}^{[l]} = \big(\boldsymbol{W}^{[l+1]\top} \boldsymbol{\delta}^{[l+1]}\big) \odot g'\big(\boldsymbol{z}^{[l]}\big).$$

The transpose sends each error back along the connections it came through, weighted by the same weights. Unit j of layer l collects $\sum_i w_{ij}^{[l+1]} \delta_i^{[l+1]}$ from the units it feeds and multiplies it by its own slope. A ReLU unit that was off ($z \le 0$) has slope 0 and gets no error.

Gradients. For every layer,

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{[l]}} = \boldsymbol{\delta}^{[l]}\, \boldsymbol{a}^{[l-1]\top}, \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{[l]}} = \boldsymbol{\delta}^{[l]}.$$

Entry by entry, $\partial \mathcal{L} / \partial w_{jk}^{[l]} = \delta_j^{[l]}\, a_k^{[l-1]}$: the error of the unit the weight feeds times the activity of the unit it comes from. A weight whose input was 0 gets a gradient of 0 on this example.

Update. Every weight and bias takes a step against its gradient,

$$\boldsymbol{W}^{[l]} \leftarrow \boldsymbol{W}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{[l]}}, \qquad \boldsymbol{b}^{[l]} \leftarrow \boldsymbol{b}^{[l]} - \eta \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{[l]}}.$$

A weight whose gradient is negative therefore increases.

Why it works. The loss depends on a weight only through a chain of quantities, for example $w_{11}^{[1]} \to z_1^{[1]} \to a_1^{[1]} \to z^{[2]} \to \mathcal{L}$. The chain rule says that the derivative along such a chain is the product of the local slopes, each evaluated at the values of the forward pass. Backpropagation multiplies these slopes starting from the loss and moving toward the input. The product from the loss back to $\boldsymbol{z}^{[l]}$ is $\boldsymbol{\delta}^{[l]}$. Every weight into layer l, and every earlier layer, shares this product, so it is computed once and reused. Each weight then needs only one more multiplication, by its input activity. One backward pass therefore costs about as much as one forward pass. Nudging each weight in turn and rerunning the network would cost one forward pass per weight.

Worked example: a 2-2-1 network

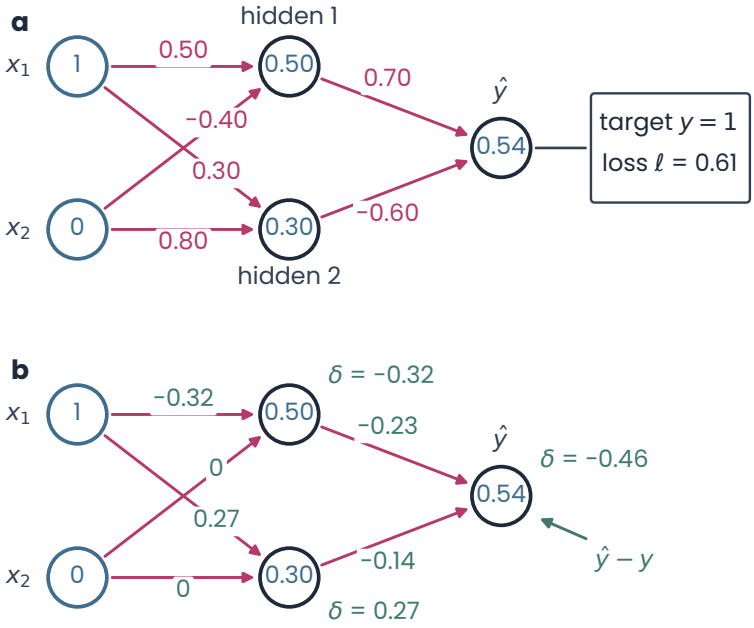
The lecture's example is the smallest network that can compute XOR: two inputs, two ReLU hidden units, one sigmoid output, and the cross-entropy loss. We use one training

example, $\boldsymbol{x} = (1, 0)$, for which XOR is 1, so $y = 1$. The biases start at 0, and the starting weights are

$$\boldsymbol{W}^{[1]} = \begin{pmatrix} 0.50 & -0.40 \\ 0.30 & 0.80 \end{pmatrix}, \qquad \boldsymbol{W}^{[2]} = \begin{pmatrix} 0.70 & -0.60 \end{pmatrix}.$$

Row 1 of $\boldsymbol{W}^{[1]}$ holds the weights into hidden unit 1: 0.50 from x_1 and -0.40 from x_2. Row 2 holds the weights into hidden unit 2: 0.30 from x_1 and 0.80 from x_2. Numbers are shown rounded to two decimals, and each step uses the unrounded values from the step before.

The figure shows the network with these weights: (a) the forward pass of Step 1, and (b) the errors and gradients of Steps 2 to 4.



Step 1: forward pass

$$\boldsymbol{z}^{[1]} = \boldsymbol{W}^{[1]}\boldsymbol{x} = \begin{pmatrix} 0.50 \cdot 1 - 0.40 \cdot 0 \\ 0.30 \cdot 1 + 0.80 \cdot 0 \end{pmatrix} = \begin{pmatrix} 0.50 \\ 0.30 \end{pmatrix}, \qquad \boldsymbol{a}^{[1]} = \mathrm{ReLU}(\boldsymbol{z}^{[1]}) = \begin{pmatrix} 0.50 \\ 0.30 \end{pmatrix}.$$

Both hidden units are on, so the ReLU passes their sums unchanged. The output is then

$$z^{[2]} = 0.70 \cdot 0.50 - 0.60 \cdot 0.30 = 0.17, \qquad \hat{y} = \sigma(0.17) = 0.54, \qquad \mathcal{L} = -\log 0.542 = 0.61.$$

Step 2: error at the output

$\delta^{[2]} = \hat{y} - y = 0.54 - 1 = -0.46$. It is negative: $z^{[2]}$ should have been higher.

Step 3: error sent backward

Both hidden units were on, so $g'(\boldsymbol{z}^{[1]}) = (1, 1)$ and

$$\boldsymbol{\delta}^{[1]} = (\boldsymbol{W}^{[2]\top}\delta^{[2]}) \odot g'(\boldsymbol{z}^{[1]}) = \begin{pmatrix} 0.70 \cdot (-0.458) \\ -0.60 \cdot (-0.458) \end{pmatrix} \odot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -0.32 \\ 0.27 \end{pmatrix}.$$

Each hidden unit gets its own error, with its own sign. Unit 1 excites the output, so it should have been more active. Unit 2 inhibits the output, so it should have been less active.

Step 4: gradients

Each gradient is error times activity:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{[2]}} = \delta^{[2]}\, \boldsymbol{a}^{[1]\top} = -0.46\,(0.50,\ 0.30) = (-0.23,\ -0.14), \qquad \frac{\partial \mathcal{L}}{\partial b^{[2]}} = -0.46,$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{[1]}} = \boldsymbol{\delta}^{[1]} \boldsymbol{x}^{\top} = \begin{pmatrix} -0.32 \\ 0.27 \end{pmatrix} (1,\ 0) = \begin{pmatrix} -0.32 & 0 \\ 0.27 & 0 \end{pmatrix}, \qquad \frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{[1]}} = \begin{pmatrix} -0.32 \\ 0.27 \end{pmatrix}.$$

The two weights from x_2 get a gradient of 0, because $x_2 = 0$ on this example.

Step 5: one update

With $\eta = 1$, subtract each gradient from its weight or bias:

$$\boldsymbol{W}^{[1]} = \begin{pmatrix} 0.82 & -0.40 \\ 0.03 & 0.80 \end{pmatrix}, \quad \boldsymbol{b}^{[1]} = \begin{pmatrix} 0.32 \\ -0.27 \end{pmatrix}, \quad \boldsymbol{W}^{[2]} = \begin{pmatrix} 0.93 & -0.46 \end{pmatrix}, \quad b^{[2]} = 0.46.$$

Run the forward pass again: $\boldsymbol{z}^{[1]} = (1.14,\ -0.25)$, so $\boldsymbol{a}^{[1]} = (1.14,\ 0)$, $z^{[2]} = 1.52$, and $\hat{y}$ rises from 0.54 to 0.82. The loss falls from 0.61 to 0.20. Hidden unit 2 is now off for this input, so on the next step it would get no error. This was one step on one example. Training repeats such steps over all the examples, many times.

Autograd

In practice, nobody writes the backward pass by hand. PyTorch records every operation of the forward pass in a computational graph. `loss.backward()` sweeps the graph in reverse, applies the chain rule at each operation, and stores the gradients of Step 4 in each parameter's `.grad`. `optimizer.step()` then performs Step 5. For the network above:

```
import torch
import torch.nn.functional as F
W1 = torch.tensor([[0.50, -0.40], [0.30, 0.80]], requires_grad=True)
W2 = torch.tensor([[0.70, -0.60]], requires_grad=True)
x, y = torch.tensor([1.0, 0.0]), torch.tensor([1.0])
z2 = W2 @ torch.relu(W1 @ x)           # forward pass (biases 0)
loss = F.binary_cross_entropy_with_logits(z2, y)
loss.backward()                        # fills W1.grad and W2.grad: Step 4
```

Backpropagation is reverse-mode automatic differentiation applied to a network: the chain rule, applied one operation at a time to the numbers computed in the forward pass.